

DSS150P Laboratory Activity 2

Evidence: Environment Setup, Source Inspection, and Pipeline Runs

1. Environment Setup

Docker Desktop running, confirming the engine is available before starting the Postgres container.

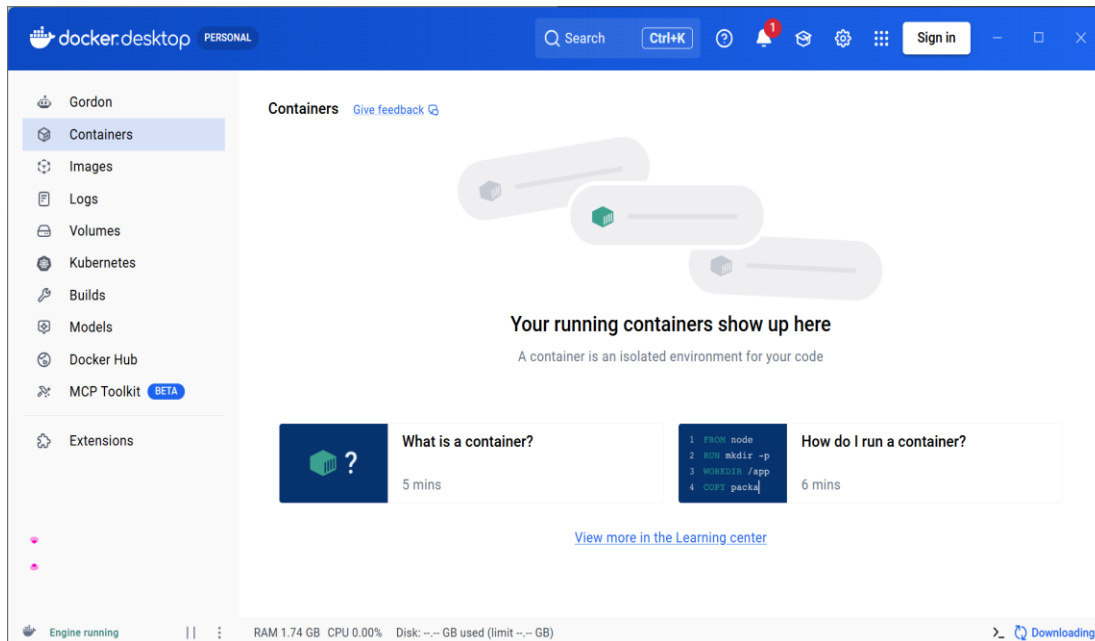


Figure 1. Docker Desktop with the engine running.

Starting the Postgres container defined in docker-compose.yml.

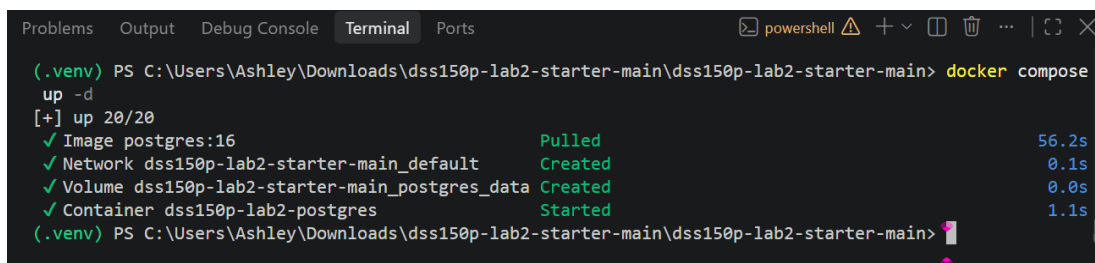


Figure 2. `docker compose up -d` successfully starting the dss150p-lab2-postgres container.

2. PostgreSQL Source Inspection (Task 1.6)

Connecting to the running Postgres container and entering the interactive psql shell.

```

-08-03T20:00:00', 'metadata': {'channel': 'partner', 'campaign': 'launch'}}}]
(.venv) PS C:\Users\Ashley\Documents\LAB ACT 2\dss150p-lab2-starter-main> ^C
(.venv) PS C:\Users\Ashley\Documents\LAB ACT 2\dss150p-lab2-starter-main> docker exec -it dss150p-lab2-postgres psql -U dss150p -d dss150p
psql (16.15 (Debian 16.15-1.pgdg13+2))
Type "help" for help.

dss150p=#

```

Figure 3. Connecting to the support_tickets database via `docker exec -it dss150p-lab2-postgres psql`.

Inspecting the support_tickets table structure with \d support_tickets.

Column	Type	Collation	Nullable	Default
ticket_id	integer		not null	
customer_id	character varying(10)		not null	
category	character varying(40)		not null	
priority	character varying(10)		not null	
assigned_agent	character varying(80)			

Figure 4. Table schema for support_tickets, showing column names, types, and nullability.

Confirming the total row count.

```

dss150p=# \d support_tickets
dss150p=# SELECT COUNT(*) FROM support_tickets;
count
-----
    250
(1 row)

dss150p=#

```

Figure 5. SELECT COUNT(*) FROM support_tickets confirms 250 rows.

Sampling the first 10 rows in primary-key order (bounded query, per lab constraints).

ticket_id	customer_id	category	priority	assigned_agent	opened_at	resolved_at	status
1	C0246	Technical	High	J. Reyes	2026-06-19 04:00:00	2026-06-21 13:00:00	Resolved
2	C0130	Product	Medium	J. Reyes	2026-05-26 07:00:00		

Figure 6. SELECT * FROM support_tickets ORDER BY ticket_id LIMIT 10.

A follow-up bounded query confirmed 4 tickets have a NULL assigned_agent value, which is documented as a data-quality finding in the profiling report.

3. REST API: Server and Pagination (Task 1.5)

Starting the local paginated REST API server.

```
PS C:\Users\Ashley\Documents\LAB ACT 2\dss150p-lab2-starter-main> git commit -m "Task 1.4: profile products.parquet"
[master f394d58] Task 1.4: profile products.parquet
 1 file changed, 10 insertions(+), 3 deletions(-)
PS C:\Users\Ashley\Documents\LAB ACT 2\dss150p-lab2-starter-main> python src/local_api_server.py
API: http://127.0.0.1:8000/api/events?page=1&per_page=20
█
```

Figure 7. Local API server running and listening on `http://127.0.0.1:8000/api/events`.

Retrieving page 1 (10 records) manually, confirming pagination fields (page, per_page, total, has_more, next_page):

```
{'page': 1, 'per_page': 10, 'total': 122, 'has_more': True, 'next_page': 2,
 'items': [
   {'event_id': 'E0001', 'customer_id': 'C0024', 'event_type': 'page_view',
    'amount': 4210.54, 'updated_at': '2026-08-01T11:00:00',
    'metadata': {'channel': 'partner', 'campaign': 'none'}},
   ... (10 items total)
 ]}
```

Retrieving page 2, confirming the page counter advances and additional unique events are returned (total=122 across all pages, meaning page 1 alone would be incomplete):

```
{'page': 2, 'per_page': 10, 'total': 122, 'has_more': True, 'next_page': 3,
 'items': [
   {'event_id': 'E0011', 'customer_id': 'C0162', 'event_type': 'checkout',
    'amount': 260.36, 'updated_at': '2026-08-02T17:00:00',
    'metadata': {'channel': 'web', 'campaign': 'none'}},
   ... (10 items total)
 ]}
```

4. First Ingestion Run (Task 3.1-3.4)

Running the ingestion pipeline for the first time from a clean state. Files are copied to `raw/files/` with a SHA-256 manifest, and API events are paginated, deduplicated by `event_id` (keeping the newest `updated_at`), written to `raw/api/events.jsonl`, and the watermark is advanced only after the write succeeds.

```
Ingested customers.csv (sha256=4229a20f...)
Ingested orders.json (sha256=7d303788...)
Ingested products.parquet (sha256=cf3b6afd...)
Starting watermark: None
Fetched page 1: 20 records (has_more=True)
Fetched page 2: 20 records (has_more=True)
Fetched page 3: 20 records (has_more=True)
Fetched page 4: 20 records (has_more=True)
Fetched page 5: 20 records (has_more=True)
Fetched page 6: 20 records (has_more=True)
Fetched page 7: 2 records (has_more=False)
Total records read: 122
Records after dedup: 120 (duplicates removed: 2)
Wrote 120 new records to raw/api/events.jsonl
```

```
Watermark advanced to: 2026-08-21T10:00:00
```

```
Run a3b853c5-348d-4e5d-b82e-bdeee906e25e completed successfully.
```

5. Idempotent Rerun (Task 3.6)

Rerunning the pipeline immediately afterward, with no changes to the source data. File hashes are unchanged so no files are re-copied, and the watermark is already at the latest value so zero new API records are fetched or written:

```
Skipping customers.csv: already ingested (hash unchanged)
Skipping orders.json: already ingested (hash unchanged)
Skipping products.parquet: already ingested (hash unchanged)
Starting watermark: 2026-08-21T10:00:00
Fetched page 1: 0 records (has_more=False)
Total records read: 0
Records after dedup: 0 (duplicates removed: 0)
No new records to write. Watermark unchanged.
```

```
Run completed successfully.
```

This behavior was additionally verified through a clean-room test: raw/ and state/ were deleted entirely and the pipeline was rerun from scratch, producing the identical final result (same file hashes, 122 records read, 120 after dedup, watermark advanced to 2026-08-21T10:00:00), confirming the pipeline is fully reproducible.

6. Failure Test and Recovery (Task 3.7)

The local API server was intentionally stopped (Ctrl+C) to simulate an outage, and the pipeline was run again:

```
Skipping customers.csv: already ingested (hash unchanged)
Skipping orders.json: already ingested (hash unchanged)
Skipping products.parquet: already ingested (hash unchanged)
Starting watermark: 2026-08-21T10:00:00
```

```
Run c5ae6571-e029-4f72-9d58-bc5ab1c9c38a FAILED:
requests.exceptions.ConnectionError: HTTPConnectionPool
(host='127.0.0.1', port=8000): Max retries exceeded
(Caused by NewConnectionError: [WinError 10061] No connection
could be made because the target machine actively refused it)
```

The watermark file was checked immediately before and after this failed run, and remained unchanged at 2026-08-21T10:00:00 in both cases, confirming the pipeline never advances state on failure. The failure was also logged as a FAILED row in outputs/pipeline_run_log.csv with the full error message captured.

The API server was then restarted, and the pipeline was run once more, completing successfully and confirming recovery:

```
Skipping customers.csv: already ingested (hash unchanged)
Skipping orders.json: already ingested (hash unchanged)
Skipping products.parquet: already ingested (hash unchanged)
Starting watermark: 2026-08-21T10:00:00
Fetched page 1: 0 records (has_more=False)
Total records read: 0
Records after dedup: 0 (duplicates removed: 0)
No new records to write. Watermark unchanged.
```

Run 891db69b-0ee2-4b07-9453-bf9fd05c8710 completed successfully.

7. Automated Validation

Running the validation script against the raw outputs confirms all automated checks pass: expected raw files exist, event_id values are unique after deduplication, required fields (including _ingested_at and _source) are present on every record, updated_at values are all parseable, and the persisted watermark matches the maximum updated_at value in the raw output.

VALIDATION PASSED: all checks succeeded